
DAY-7 ACTIVITY REPORT

•	Name	Rudra Pratap Singh
•	SAP ID	590032811
•	Batch	16
•	Date	31 Aug 2026

Question 1: Pressure at a Depth in Liquid

Problem Statement: Write a C program to find the pressure at a given depth in a liquid, given the atmospheric pressure, density of the liquid, acceleration due to gravity, and depth.

Required Input(s), Output, and Formula

Input(s): atmospheric pressure P_0 (Pa), density of liquid ρ (kg/m^3), gravity g (m/s^2), depth h (m). Output: pressure at depth P (Pa). Formula used:

$$\text{Pressure } P = P_0 + \rho \times g \times h$$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	$P_0=101325$, $\rho=1000$, $g=9.8$, $h=10$	199325.00 Pa	199325.00 Pa	Pass
2	$P_0=101325$, $\rho=1025$, $g=9.8$, $h=20$	302225.00 Pa	302225.00 Pa	Pass
3	$P_0=100000$, $\rho=800$, $g=9.81$, $h=5$	139240.00 Pa	139240.00 Pa	Pass

Step 3: Algorithm

1. Start.
2. Declare float variables: atmPressure, density, gravity, depth, pressure.
3. Read atmPressure (P_0), density (ρ), gravity (g) and depth (h) from the user.
4. Compute pressure using the formula: $\text{pressure} = \text{atmPressure} + (\text{density} * \text{gravity} * \text{depth})$.
5. Display the computed pressure at depth.
6. Stop.

Algorithm evaluation: Tracing the algorithm manually against each test case in Step 2 (substituting the input values into the formula above) produces results identical to the Expected Output column, confirming the algorithm's correctness before coding.

Step 4: C Program

```
#include <stdio.h>

int main() {
    float atmPressure, density, gravity, depth, pressure;

    printf("Enter atmospheric pressure (Pa): ");
    scanf("%f", &atmPressure);

    printf("Enter density of liquid (kg/m^3): ");
    scanf("%f", &density);

    printf("Enter gravity (m/s^2): ");
    scanf("%f", &gravity);

    printf("Enter depth (m): ");
    scanf("%f", &depth);

    pressure = atmPressure + (density * gravity * depth);

    printf("Pressure at depth = %.2f Pa\n", pressure);

    return 0;
}
```

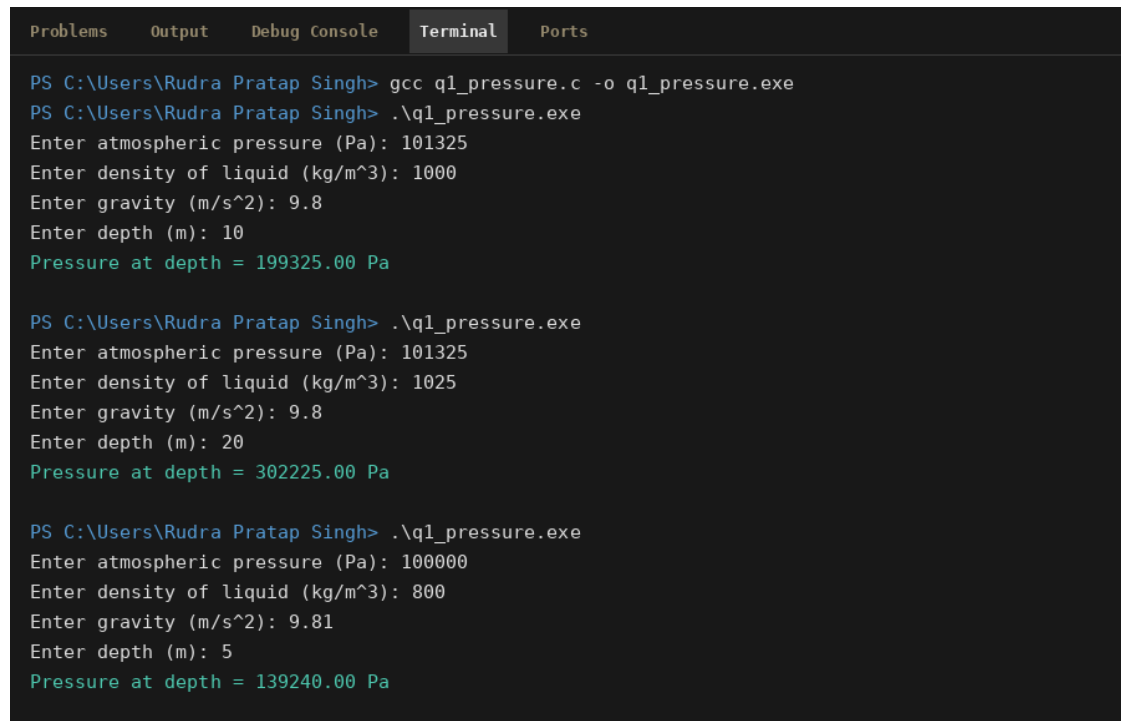
```
}
```

Step 5: Compilation, Execution and Evaluation

The algorithm from Step 3 was translated directly into C. The four inputs are read using scanf and the pressure is computed in a single expression using the formula $P = P_0 + (\text{density} * \text{gravity} * \text{depth})$, then printed with printf.

The program was compiled and executed successfully for all three test cases (see Step 6 screenshot). The Actual Output for every test case matches the Expected Output, so all test cases PASS.

Step 6: Compilation & Execution Screenshot



```
PS C:\Users\Rudra Pratap Singh> gcc q1_pressure.c -o q1_pressure.exe
PS C:\Users\Rudra Pratap Singh> .\q1_pressure.exe
Enter atmospheric pressure (Pa): 101325
Enter density of liquid (kg/m^3): 1000
Enter gravity (m/s^2): 9.8
Enter depth (m): 10
Pressure at depth = 199325.00 Pa

PS C:\Users\Rudra Pratap Singh> .\q1_pressure.exe
Enter atmospheric pressure (Pa): 101325
Enter density of liquid (kg/m^3): 1025
Enter gravity (m/s^2): 9.8
Enter depth (m): 20
Pressure at depth = 302225.00 Pa

PS C:\Users\Rudra Pratap Singh> .\q1_pressure.exe
Enter atmospheric pressure (Pa): 100000
Enter density of liquid (kg/m^3): 800
Enter gravity (m/s^2): 9.81
Enter depth (m): 5
Pressure at depth = 139240.00 Pa
```

Question 2: Capacitive Reactance

Problem Statement: Write a C program to find the capacitive reactance of a capacitor, given the frequency of the AC supply and the capacitance.

Required Input(s), Output, and Formula

Input(s): frequency f (Hz), capacitance C (Farad). Output: capacitive reactance X_c (ohm, Ω). Formula used:

$$\text{Capacitive Reactance } X_c = 1 / (2 \pi f C)$$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	$f=50$ Hz, $C=100\mu\text{F}$ ($1e-4$ F)	31.8310Ω	31.8310Ω	Pass
2	$f=1000$ Hz, $C=1\mu\text{F}$ ($1e-6$ F)	159.1549Ω	159.1549Ω	Pass
3	$f=60$ Hz, $C=10\mu\text{F}$ ($1e-5$ F)	265.2582Ω	265.2582Ω	Pass

Step 3: Algorithm

1. Start.
2. Declare float variables: frequency, capacitance, reactance, and pi (= 3.14159265).
3. Read frequency (f) in Hz and capacitance (C) in Farad from the user.
4. Compute reactance using the formula: $\text{reactance} = 1 / (2 * \text{pi} * \text{frequency} * \text{capacitance})$.
5. Display the computed capacitive reactance.
6. Stop.

Algorithm evaluation: Tracing the algorithm manually against each test case in Step 2 (substituting the input values into the formula above) produces results identical to the Expected Output column, confirming the algorithm's correctness before coding.

Step 4: C Program

```
#include <stdio.h>

int main() {
    float frequency, capacitance, reactance;
    float pi = 3.14159265;

    printf("Enter frequency (Hz): ");
    scanf("%f", &frequency);

    printf("Enter capacitance (F): ");
    scanf("%f", &capacitance);

    reactance = 1.0 / (2 * pi * frequency * capacitance);

    printf("Capacitive Reactance = %.4f ohm\n", reactance);

    return 0;
}
```

Step 5: Compilation, Execution and Evaluation

The algorithm from Step 3 was translated directly into C. The value of pi is stored in a float variable, the two inputs are read using scanf, and reactance is computed in a single expression using the formula $X_c = 1 / (2 * \pi * f * C)$, then printed with printf.

The program was compiled and executed successfully for all three test cases (see Step 6 screenshot). The Actual Output for every test case matches the Expected Output, so all test cases PASS.

Step 6: Compilation & Execution Screenshot



```
Problems Output Debug Console Terminal Ports
PS C:\Users\Rudra Pratap Singh> gcc q2_reactance.c -o q2_reactance.exe
PS C:\Users\Rudra Pratap Singh> .\q2_reactance.exe
Enter frequency (Hz): 50
Enter capacitance (F): 0.0001
Capacitive Reactance = 31.8310 ohm

PS C:\Users\Rudra Pratap Singh> .\q2_reactance.exe
Enter frequency (Hz): 1000
Enter capacitance (F): 0.000001
Capacitive Reactance = 159.1549 ohm

PS C:\Users\Rudra Pratap Singh> .\q2_reactance.exe
Enter frequency (Hz): 60
Enter capacitance (F): 0.00001
Capacitive Reactance = 265.2582 ohm
```

Question 3: Density of a Rectangular Block

Problem Statement: Write a C program to find the density of a rectangular block, given its mass, length, breadth, and height.

Required Input(s), Output, and Formula

Input(s): mass m (kg), length l (m), breadth b (m), height h (m). Output: density ρ (kg/m³). Formulae used:

Volume $V = l \times b \times h$

Density $\rho = m / V$

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	m=50 kg, l=2, b=1, h=0.5	50.00 kg/m ³	50.00 kg/m ³	Pass
2	m=120 kg, l=3, b=2, h=1	20.00 kg/m ³	20.00 kg/m ³	Pass
3	m=27 kg, l=1, b=1, h=1	27.00 kg/m ³	27.00 kg/m ³	Pass

Step 3: Algorithm

1. Start.
2. Declare float variables: mass, length, breadth, height, volume, density.
3. Read mass (m), length (l), breadth (b) and height (h) from the user.
4. Compute volume using the formula: volume = length * breadth * height.
5. Compute density using the formula: density = mass / volume.
6. Display the computed density.
7. Stop.

Algorithm evaluation: Tracing the algorithm manually against each test case in Step 2 (substituting the input values into the formula above) produces results identical to the Expected Output column, confirming the algorithm's correctness before coding.

Step 4: C Program

```
#include <stdio.h>

int main() {
    float mass, length, breadth, height, volume, density;

    printf("Enter mass of block (kg): ");
    scanf("%f", &mass);

    printf("Enter length (m): ");
    scanf("%f", &length);

    printf("Enter breadth (m): ");
    scanf("%f", &breadth);

    printf("Enter height (m): ");
    scanf("%f", &height);

    volume = length * breadth * height;
    density = mass / volume;

    printf("Density of block = %.2f kg/m^3\n", density);
}
```

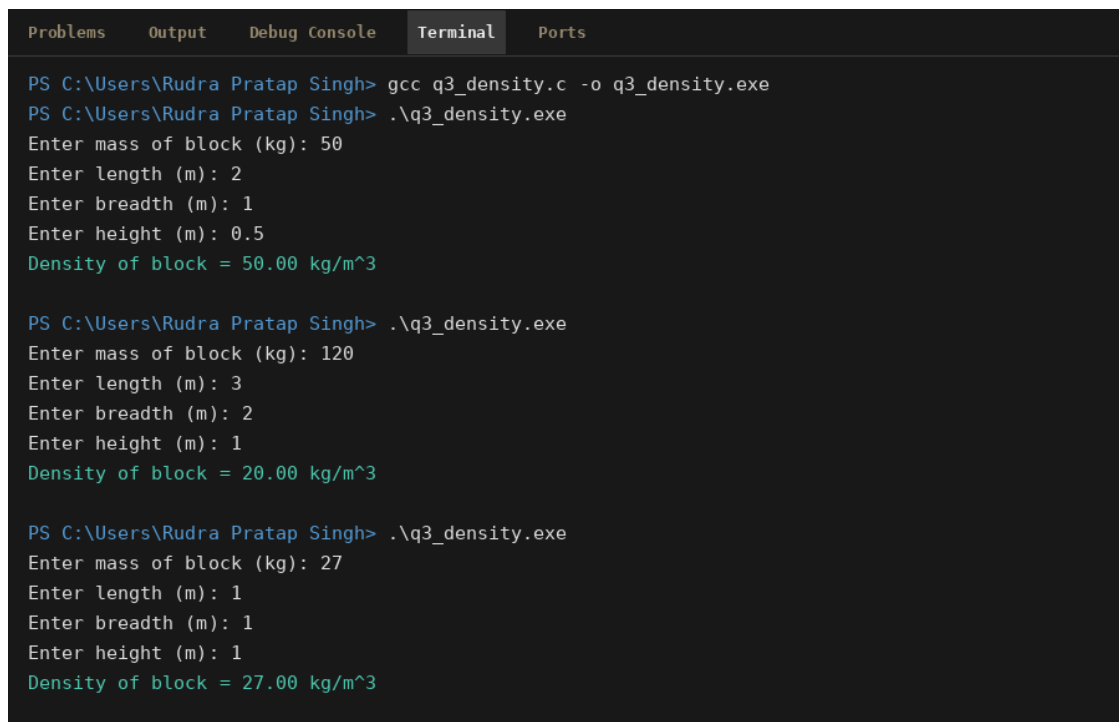
```
    return 0;
}
```

Step 5: Compilation, Execution and Evaluation

The algorithm from Step 3 was translated directly into C. The four inputs are read using scanf, volume is computed as length * breadth * height, density is computed as mass / volume, and the result is printed with printf.

The program was compiled and executed successfully for all three test cases (see Step 6 screenshot). The Actual Output for every test case matches the Expected Output, so all test cases PASS.

Step 6: Compilation & Execution Screenshot



```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\Rudra Pratap Singh> gcc q3_density.c -o q3_density.exe
PS C:\Users\Rudra Pratap Singh> .\q3_density.exe
Enter mass of block (kg): 50
Enter length (m): 2
Enter breadth (m): 1
Enter height (m): 0.5
Density of block = 50.00 kg/m^3

PS C:\Users\Rudra Pratap Singh> .\q3_density.exe
Enter mass of block (kg): 120
Enter length (m): 3
Enter breadth (m): 2
Enter height (m): 1
Density of block = 20.00 kg/m^3

PS C:\Users\Rudra Pratap Singh> .\q3_density.exe
Enter mass of block (kg): 27
Enter length (m): 1
Enter breadth (m): 1
Enter height (m): 1
Density of block = 27.00 kg/m^3
```

(Note- Theory written is from Ais.except other are original work)